

Enhancing Java Web Application Security: Injection Vulnerability Detection via Interprocedural Analysis and Deep Learning

Bing Zhang¹, Xu Zhi¹, Meng Wang¹, Rong Ren¹, and Jun Dong²

Abstract—Injection attacks exploit vulnerabilities in how applications handle user input, allowing malicious code to infiltrate the execution environment of web applications, leading to severe consequences, such as data leaks and system crashes. Traditional dynamic and static detection methods suffer from limitations in manual rule or pattern design and intraprocedural analysis, lacking the capability to automatically learn complex features. Meanwhile, deep learning models encounter challenges, such as feature redundancy and inefficiency, in processing long code sequences. Here, we propose a prototype for detecting Injection Vulnerabilities in Java web applications based on Interprocedural analysis and the bidirectional encoder representations from transformers BERT-BiLSTM-CRF model (IVIB), effectively transforming vulnerability detection into text sequence annotation. IVIB employs interprocedural analysis to trace complete program data flow, control flow, method and class dependencies, reducing redundancy through a system dependency graph. Then, we develop intermediate language representation rules and conversion mechanisms specifically for Java programs, symbolically representing code snippets and annotating them to construct a corpus. IVIB achieves remarkable results, with over 96% accuracy, precision, recall, and F1-score in binary classification, surpassing other state-of-the-art models in multiclassification performance. Evaluation on real-world projects demonstrates IVIB's effectiveness, detecting 28 vulnerabilities out of 30 vulnerable slices with low false positives and no false negatives.

Index Terms—BERT-BiLSTM-CRF, injection vulnerability, intermediate slice language (ISL), system dependency graph, vulnerability detection.

I. INTRODUCTION

ACCORDING to OWASP Top 10-2021 report [1], injection vulnerabilities are considered one of the most critical security risks in web applications. Common injection attacks

include structured query language (SQL) injection, cross-site scripting (XSS) vulnerabilities, command injection vulnerabilities, and XPath injection vulnerabilities, all of which exploit the same weakness: data flowing from a source controlled by the user of a web application to a receiver that processes this data without proper validation, potentially triggering dangerous operations. While web applications can be implemented using various programming languages, according to JetBrains' report in 2022 [2], Java remains one of the primary languages for developing web systems. Effectively detecting and controlling injection vulnerabilities in Java web applications can significantly enhance their security, protecting user information and system stability.

Traditional injection vulnerability detection methods are mainly divided into static analysis, dynamic testing, and hybrid analysis. Static analysis primarily detects vulnerabilities by analyzing source code. Due to the inability to execute programs, it is often inefficient to identify vulnerable code solely through intraprocess code analysis [3], which may result in false positives and false negatives. In addition, static analysis heavily relies on the experience of security experts [4], [5], such as the need to manually establish attack pattern libraries [4] and write different regular expressions based on vulnerability characteristics [5]. Dynamic testing simulates the actual runtime environment by executing programs, allowing access to various runtime contexts/data that static analysis cannot obtain for vulnerability analysis [6]. However, vulnerabilities may remain undetected due to low code coverage [7], [8], and incomplete rule databases can affect the effectiveness of dynamic testing in practical applications [9]. Hybrid analysis combines the advantages of both methods but may suffer from issues such as feature extraction [10] and the curse of dimensionality [11].

The application of deep learning in vulnerability detection has attracted widespread attention [12], [13], [14]. Research on various types of vulnerabilities is abundant, especially for buffer overflow vulnerabilities (CWE-119) [15], [16], resource management errors (CWE-399) [17], [18], and injection vulnerabilities (CWE-89) [19], [20]. However, it is noted that there is relatively limited research on using deep learning models for detecting code injection vulnerabilities in the Java language within existing literature. In addition, most studies [15], [17], [21], including those employing static analysis [3], focus on intraprocedural analysis, examining code vulnerabilities strictly within individual functions or methods, with limited emphasis on interprocedural analysis. This intraprocedural focus has certain limitations, such as a lack of insight into the program's broader structure and the inability to capture essential relationships between function calls. Interprocedural analysis addresses

Received 30 June 2024; revised 3 November 2024; accepted 19 December 2024. Date of publication 7 January 2025; date of current version 4 September 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62376240, in part by the S&T Program of Hebei under Grant 226Z0701G, Grant 236Z0702G, and Grant 236Z0304G, in part by the Natural Science Foundation of Hebei Province under Grant F2022203026, Grant F2022203089, and Grant F2023203026, in part by Basic Innovation Research Cultivation Program of Yanshan University under Grant 2024LGZD004, in part by the Science and Technology Project of Hebei Education Department under Grant BJK2022029, and in part by the Innovation Capability Improvement Plan Project of Hebei Province under Grant 22567637H. Associate Editor: J. Zhou. (Corresponding author: Rong Ren.)

The authors are with the School of Information Science and Engineering, Yanshan University, Qinhuangdao 066004, China, and also with the Key Laboratory for Software Engineering of Hebei Province, Qinhuangdao 066004, China (e-mail: bingzhang@ysu.edu.cn; dixu1999@163.com; wangmeng@163.com; renrongysu@hotmail.com; dongjun@ysu.edu.cn).

Digital Object Identifier 10.1109/TR.2024.3521381

these limitations by considering interactions across functions, providing a more comprehensive view of code behavior and potential vulnerabilities.

Compared to traditional vulnerability detection methods like static analysis [4], [22] and dynamic analysis [6], [7], deep learning can automatically learn features [23], [24], [25], [26], but it may produce redundant features, such as common variable or function names, increasing model complexity and training time. This highlights the significance of intermediate language conversion methods, which simplify code complexity without losing critical information, thereby enhancing model efficiency and generalization. Among deep learning models, bidirectional long short-term memory (BiLSTM) is regarded as one of the best for vulnerability detection [27], as it integrates forward and backward sequence information and addresses gradient explosion or vanishing issues found in traditional long short-term memory (LSTM) models [28], [29]. However, BiLSTM can struggle with long-term dependencies in extremely lengthy sequences [30]. To mitigate this, feature vectorization of long sequences is common. While Word2Vec [31] segments code into word-level representations, it may not capture context deeply [32]. Sent2Vec [33] converts text into vector representations independently, potentially losing contextual information. In contrast, the conditional random field (CRF) model [34] effectively considers dependencies between contextual information and adjacent code features. In addition, some studies [35], [36], [37] have leveraged the bidirectional encoder representations from transformers (BERT) model for feature representation due to its strong performance in natural language tasks.

Deep learning has also been applied in various ways to Java vulnerabilities. Feng et al. [38] constructed interprocedural control flow graphs from Java bytecode using graph neural networks, but their approach may overlook critical control dependencies and data transformations. Chi et al. [39] and Nguyen et al. [40] focused on vulnerability fixing in Java programs, while Song and Ding [41] examined the robustness of deep learning models against adversarial attacks with their tool Milo. Liu et al. [42] proposed GraphSearchNet to enhance source code search by leveraging graph neural networks for rich semantic capture. However, these methods do not specifically address injection vulnerabilities in Java code.

To better capture the characteristic information of code injection vulnerabilities in the Java language and improve the accuracy and efficiency of vulnerability detection, we propose a novel prototype, Injection Vulnerabilities in Java web applications based on Interprocedural analysis and the BERT-BiLSTM-CRF model (IVIB), for detecting injection vulnerabilities in Java web applications, based on interprocedural analysis and BERT-BiLSTM-CRF.

First, IVIB leverages interprocedural analysis and taint analysis to cross function and module boundaries, accurately tracking data and control flow. It preserves the complete code paths related to vulnerability features throughout the process. Compared with intraprocedural analysis, interprocedural analysis excels in covering a broader range of code. We focus solely on potential vulnerability code snippets from source to sinks, significantly reducing redundancy.

Next, we define a set of novel Intermediate Slice Language (ISL) representation rules tailored for Java, transforming and simplifying vulnerability code slices to avoid excessive accumulation of irrelevant or similar features while retaining more valid

ones. Token-level feature representation effectively addresses granularity issues. We define a dynamic attribute state set comprising 11 states for code paths, annotate the code segments following intermediate representation, and build a corpus with a significant number of samples.

Finally, we devise a deep learning model that combines BERT with BiLSTM-CRF for feature extraction and classification on the constructed corpus to achieve vulnerability detection. BERT captures global contextual information and handles long-term dependencies in sequences effectively. BiLSTM-CRF captures local sequence patterns efficiently. The combination enhances the model's sensitivity and accuracy in understanding and categorizing potential vulnerability points in the code, significantly boosting its performance.

To the best of the authors' knowledge, this work is the first to extend injection vulnerability detection to Java using interprocedural analysis and deep learning, surpassing the hypertext preprocessor (personal home page) (PHP)-specific focus of Medeiros et al. [43], which lacks the ability to process more complex constructs in PHP, such as objects. To address Java's object-oriented complexity, ISL we defined can represent classes and objects effectively. Our work extends Medeiros's work across platforms, and expands detection beyond SQL injection to cover multiple vulnerabilities, including XSS, Command Injection, and XPath Injection, making this approach well-suited for Java-based applications. The main contributions of this article are summarized as follows.

- 1) We proposed a novel prototype, IVIB (shared online¹), for detecting injection vulnerabilities in Java web applications. By leveraging interprocedural analysis and the advanced architecture of BERT-BiLSTM-CRF, IVIB markedly enhances the efficiency of identifying injection attacks.
- 2) We applied an interprocedural slicing analysis method combined with taint analysis for an in-depth examination of the source code. This approach retains code paths associated with vulnerability features, focusing specifically on relevant code segments to reduce redundancy and enhance interprocedural analysis of object and method calls unique to Java.
- 3) We developed a novel intermediate language representation tailored to Java, capturing the distinctive features of Java's objects and methods. This representation minimizes irrelevant or repetitive features. In addition, we defined a multistate semantic annotation model for the intermediate language code sequences and built a dedicated corpus to support precise vulnerability detection.
- 4) We assessed IVIB by using two popular Java-based test sets. When compared with four other state-of-the-art models, IVIB exhibited superior performance in multiclass classification. In addition, out of 42 slices derived from four real-world projects, IVIB successfully identified a total of 28 injection vulnerabilities and 7 benign slices, with low false positives and zero false negatives.

The rest of this article is organized as follows. Section II explains the injections in web applications. Section III provides a detailed description of our method. Section IV presents the experimental evaluation. Section V discusses related work. Section VI delves into a discussion of our work. Finally, Section VII concludes this article.

¹[Online]. Available: <https://github.com/KLSEHB/IVIB/tree/main>

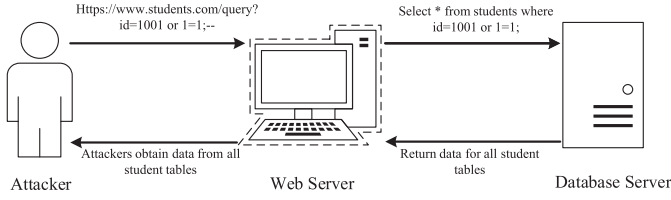


Fig. 1. SQL injection attack process.

II. INJECTIONS IN WEB APPLICATIONS

The significant injection vulnerabilities in web applications can be summarized as SQL injection, XSS injection, Command injection, and XPath injection, described in detail as follows.

A. SQL Injection

SQL injection occurs when unchecked user input is passed to execute commands in the backend database. In Fig. 1, the process of how attackers exploit the SQL injection vulnerability to retrieve all student data from the application database is described. Attackers achieve this by injecting crafted malicious input (`1001 or 1=1;--`) into the backend code to carry out the data retrieval.

B. XSS Injection

XSS injection is a common web security vulnerability that allows attackers to execute malicious scripts in a victim's browser. Attackers can steal user information, session credentials, or manipulate page content by inserting malicious code, such as JavaScript, into web pages. Here is an example of XSS injection, assuming there is a vulnerable input field mentioned in the code.

```
<input type="text" id="inputField">
<script>
var userInput =
document.getElementById('inputField').value;
document.write('User input: ' + userInput);
</script>
```

In this example, if an attacker inputs malicious script code, such as `<script> alert('XSS Attack!') </script>`, into the input field, when other users access the page containing this malicious code, a pop-up alert saying "XSS Attack!" will be displayed. This demonstrates the security implications that an XSS injection vulnerability can lead to.

C. Command Injection

Command injection is a type of security vulnerability where an attacker can execute arbitrary commands on a system. This usually occurs when an application does not properly validate and sanitize user input, allowing the attacker to inject additional commands that are inadvertently executed by the system's shell or command interpreter. Here is an example of a command injection vulnerability in a practical scenario.

```
if ($_SERVER["REQUEST_METHOD"]=="POST") {
    $ip_address = $_POST['ip_address'];
    // ping IP
    $output=shell_exec("ping" . $ip_address);
    echo "<pre>" . $output . "</pre>";
}
```

Command injection input:

```
user input: 127.0.0.1; ls -la.
```

In this example, if the user input `127.0.0.1; ls -la` is not properly sanitized, it can lead to the execution of the `ls -la` command on the system, potentially revealing sensitive information.

D. XPath Injection

XPath injection is a type of security vulnerability that occurs when untrusted data are inserted into an XPath query without proper validation, allowing attackers to manipulate the query's logic and potentially access unauthorized data. In the context of XML querying, XPath injection exploits can lead to unintended data disclosure or data manipulation. Considering the following code snippet, we illustrate an XPath injection vulnerability in an XML query:

```
<library>
  <book>
    <title>Python Programming</title>
    <author>Alice</author>
    <year>2022</year>
  </book>
</library>.
```

XPath query input:

```
/library/book[author='Alice' or 1=1]
/title.
```

In this example, the XPath query contains unvalidated user input `author='Alice' or 1=1`, which alters the query's logic to retrieve all titles for books authored by "Alice" as well as all titles due to the always true condition `or 1=1`. This demonstrates how XPath injection can bypass access controls and compromise the integrity of XML data retrieval in a vulnerable system. Proper input validation and sanitization are crucial to mitigate XPath injection risks and ensure secure XML querying practices.

III. METHODS

IVIB is designed as a novel prototype to detect the injection vulnerabilities like SQL injection, XSS, command injection and XPath injection in Java web Applications. IVIB are divided into four parts: SDG modeling, program slicing, intermediate language representation and state annotation, and vulnerability detection, as shown in Fig. 2.

1) *SDG modeling*: IVIB is designed to focus on code snippets only related to vulnerabilities through interprocedural analysis. Based on abstract syntax tree (AST) nodes at the element level in code line and the relationships between code elements, control dependence graph and data dependence graph dependency edges and nodes are supplemented into the AST. In addition, relationships between different methods and classes across files are also added to form the SDG model.

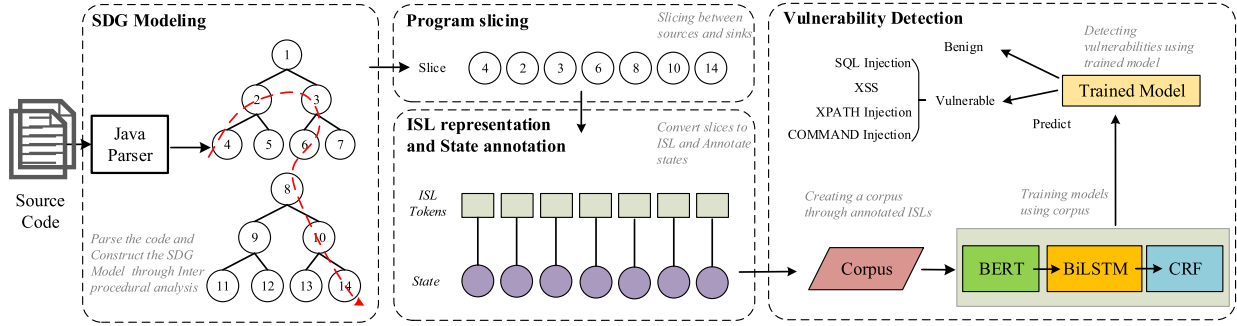


Fig. 2. Overview of the IVIB prototype.

2) *Program slicing*: Based on the SDG model and the characteristics of injection vulnerabilities, paths slices from injection points to sink points are extracted to reduce noise and improve detection efficiency, forming a collection of vulnerability slices.

3) *ISL representation and State annotation* further eliminate different representations between different code slices, unify them, and then, based on specified constraint rules, annotate the state of different operation attributes in the code snippets to form a corpus.

4) *Vulnerability detection*: Based on the corpus, an efficient BERT-BiLSTM-CRF model for Vulnerability detection and analysis is proposed.

A. SDG Modeling

Injection vulnerabilities are usually associated with direct user inputs, where user input (source) reaches the location where vulnerabilities can be exploited or triggered (sink). If in this process, user input lacks necessary checks or even some malicious injections bypass check functions, it will result in an injection vulnerability, which has very obvious characteristics. In complex Java web applications, ignoring code unrelated to injection vulnerabilities and focusing only on these code paths will enhance detection efficiency. Furthermore, user inputs may be treated as parameters for different method calls, and crossing functions and files is common, making interprocedural analysis necessary. To achieve this goal, we have automatically built a System Dependence graph model based on interprocedural global information representation.

We initially employ JavaParser [44] to parse the source code of Java web applications into an AST. As depicted in Listing 1, `LoginServlet` and `LoginDao` are two classes, with a calling relationship between them (Line 5), existing in the same package. Fig. 3 illustrates the AST structure of these two classes, with nodes in their AST represented hierarchically, indicated by green directed AST edges.

Based on the AST, we further supplemented the control dependency relationship between nodes, such as the `loginDao` object calling its `login` property, and the `getString` method of the data query result set `rs`. In Fig. 3, we represent control dependencies with blue directed edges. More importantly, there are data dependency relationships between nodes at different levels, which can tightly link methods and classes together. In Fig. 3, we use pink directed edges to represent these dependencies. Finally, to achieve interprocedural analysis, method calls are represented by orange directed edges, and class calls are

represented by brown directed edges, and the data dependency edges between procedures have also been enhanced (e.g., edge between `Param-in: username` and `username`). Here, the variables marked with `Param-in` are the function's formal parameters, and `Param-out` are the variables corresponding to the function's return values. Ultimately, this forms a SDG model that can characterize the global information of the program.

B. Program Slicing

After obtaining the global SDG, in order to extract code snippets specifically related to injection vulnerabilities, we designed a fully automatic forward and inward (FI) slicing algorithm. Forward slicing and Inward slicing are defined as follows.

- 1) *Forward slicing* is a code analysis technique aimed at extracting all subsequent code parts that may be influenced by a specific line of code. It analyzes data dependencies to trace how specific variables or states are used in the following code, helping to identify the relevant code segments that affect the program's output.
- 2) *Inward slicing* [45] is an analysis based on the context of function calls. When a function is called, the program's control flow shifts from the current execution point to the inside of that function. Inward slicing extracts all the code lines that the control flow and data flow pass through within that function. This method helps to understand the internal logic of the function and its impact on the calling context.

FI algorithm extracts paths from the SDG model between the source and sink points. For example, in Fig. 3, `Param-out: username` and `Param-out: password` represent user inputs, which are the source points, while the node `Param-in: res` with no out-degree represents the output result after a database query, which is highly likely to be the location where a vulnerability may be triggered, acting as the sink point. In contrast to the vast and complex SDG, it has been proven that there are only two paths (red dotted directed edges) between the source and sink points in Fig. 3, significantly reducing the amount of code that needs to be analyzed. We then represent these paths using dotted pink directed edges. Based on the key data dependency, the FI slicing algorithm is shown as follows.

In Algorithm 1, lines 1–9 define the main function method for FI Slicing, which traverses the nodes in the `Param-out` set of the SDG. Line 4 checks whether their sibling nodes are related to the `inputToken` (defined in Section III-C). If that is true, the

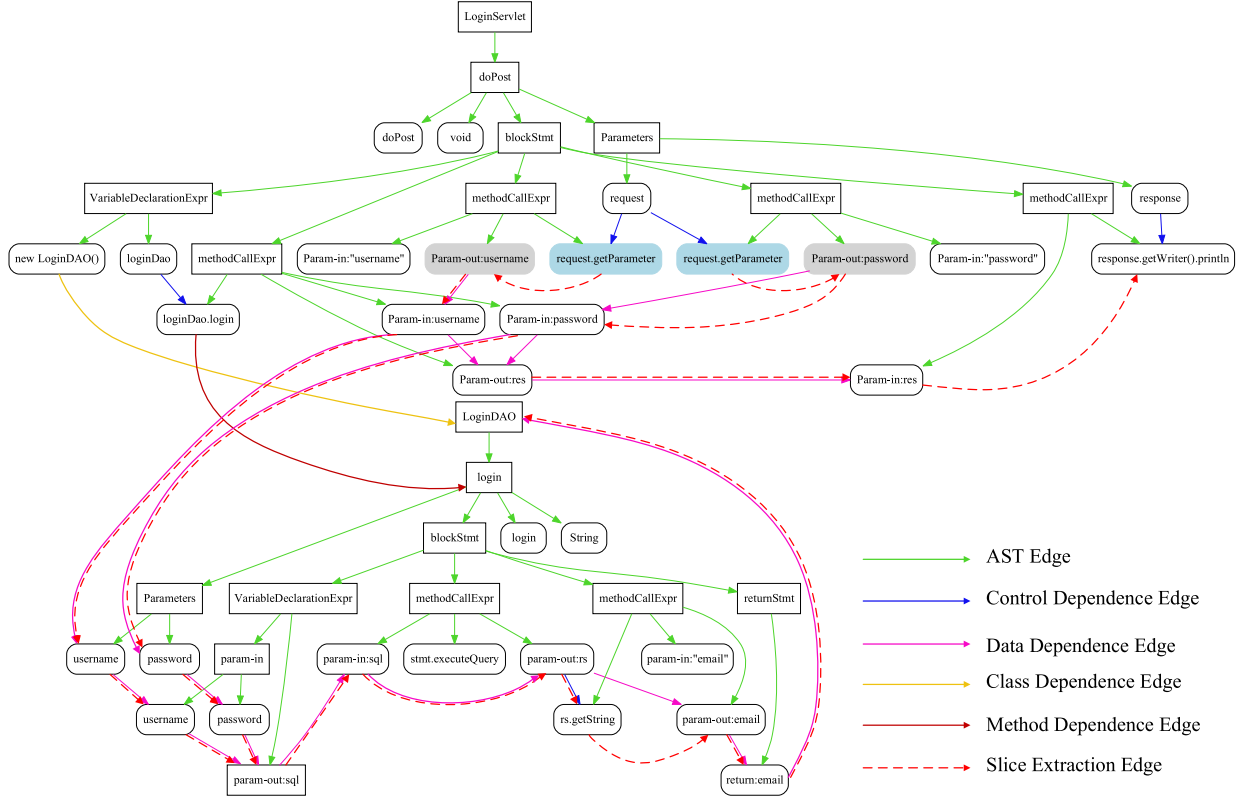


Fig. 3. Example of SDG modeling process.

```

1 package com.vul.sql.servlet;
2 import com.vul.sql.dao.LoginDAO;
3 public class LoginServlet{
4     public void doPost(HttpServletRequest request, HttpServletResponse response){
5         LoginDAO loginDao=new LoginDAO();
6         String username=
7             request.getParameter("username");
8         String password=
9             request.getParameter("password");
10        String res=
11            loginDao.login(username,password);
12        response.getWriter().println(res);
13    }
14 }
15
16 package com.vul.sql.dao;
17 public class LoginDAO{
18     public void login(String username,
19 String password){
20         ...
21         String sql="SELECT * from USERS
22 where USERNAME='"+username+"' and
23 PASSWORD='"+password+"'";
24         ResultSet rs=pstmt.executeQuery(sql);
25         String email=rs.getString("email");
26         return email;
27     }
28 }

```

Listing 1. Code snippets of Java

current node is considered an input variable, and it is set as the starting node for injecting vulnerability paths. Line 5 then uses the recursive method *FIslicingRecursive* function (lines 10–22) to extract *PVCS*.

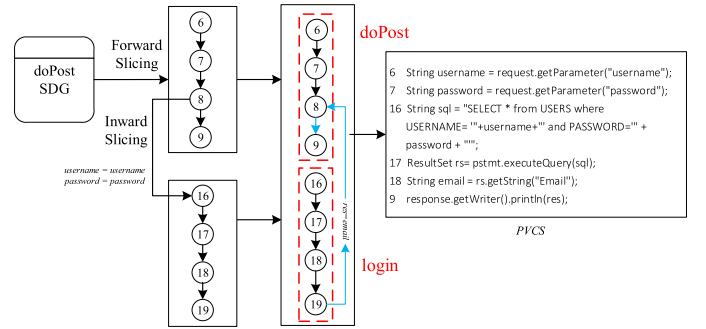


Fig. 4. SDG Modeling process for the code in Listing 1.

Within the *FIslicingRecursive* function, lines 11–13 serve as recursive exits, and line 14 indicates extracting all code lines containing the variable based on input variables to form a *forwardSlice*, with the node becoming the next node (line 15). It then checks if the node is empty and if its state changes to *Param-in*. If that is true, it indicates that the variable will be used as input for other methods, facilitating parameter passing. Subsequently, it further extracts all code lines *inwardSlice* containing the next node (line 17). Then, *forwardSlice* and *inwardSlice* are connected through parameters of different functions, and the result is assigned to *forwardSlice* (line 18). Finally, it recursively extracts all paths starting from this node (line 19), gradually expanding *PVCS* (line 20), ultimately obtaining a concise and accurate *PVCSs*.

Algorithm 1: FI Slicing Algorithm.

Input: *SDG, Param-out set, Source code*
Output: *Potential Vulnerable Code Snippets (PVCS)*

```

1: function FIslicing(SDG, Param-out set, Source code){
2:   PVCS ← []
3:   for each varName in Param-out set
4:     if varName's sibling Node is related to inputToken
       then
5:       FIslicingRecursive{varName, SDG, Source code, PVCS}
6:     endif
7:   endfor
8:   return PVCS
9: }
10: function FIslicingRecursive(varName, SDG, Source code, PVCS){
11:   if varName is null then
12:     return
13:   endif
14:   forwardSlice ← extractLinesContaining (varName)
       in Source code
15:   nextVarName ← varName.nextNode() in SDG
16:   if nextVarName is not null and nextVarName's status
       changes to Param-in then
17:     inwardSlice ← extractLinesContaining
       (nextVarName.nextNode() in SDG) in Source code
18:     forwardSlice ← mergebyParamMapping
       (forwardSlice, inwardSlice)
19:     FIslicingRecursive(nextVarName, SDG, Source code, PVCS)
20:   endif
21:   PVCS. Add (forwardSlice)
22: }
```

Using Algorithm 1, we extract the PVCSs from the code in Listing 1, which is as illustrated in Fig. 4. The numbers in the figure represent the line numbers in Listing 1. Forward slicing results in lines 6, 7, 8, and 9 of code, with line 8 indicating a function call. Through inward slicing, parameter passing based on username and password yields lines 16, 17, 18, and 19. Finally, the variable *res* from line 8 and the return value *email* from line 19 are found to be identical. By merging these results, we ultimately identify lines 6, 7, 16, 17, 18, and 19 as highly relevant to the identified vulnerability.

C. ISL Representation and State Annotation

Through Program Slicing, PVCSs are obtained from the application source code, whether it is used for learning or detection purposes. Since IVIB focuses on injection vulnerabilities, these PVCSs consist of a series of tracing instructions from user inputs (sources) to points where vulnerabilities could be triggered (sinks), which may include if statements, validation functions, etc. However, some instructions with similar functionalities may have different representations in the code, generating more features. For example, *getString* and *getQueryString* both represent retrieving input parameters, but with different function names, which are two features within a slice sequence. Therefore, we represent the instructions in the code as an ISL [43] to better trace and analyze the critical functionalities in

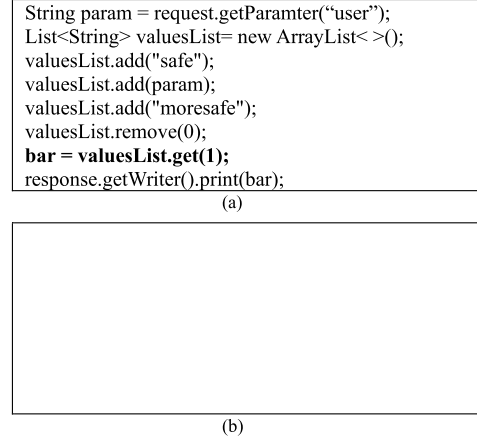


Fig. 5. Example of code operation of list variable. (a) Retrieving data from the second value. (b) Retrieving data from the first value.

the code. By using ISL, different forms of code instructions can be unified into syntactic structures with similar functionalities, thus integrating features at the code level and providing more concise code analysis capabilities.

This section first defines the ISL instructions for Java (see Section III-C1), then designs instruction transformation algorithms for PVCSs (see Section III-C2), and finally annotates the transformed instruction sequences with six categories of status labels according to specified rules (see Section III-C3), forming the corpus required for IVIB.

1) *ISL Creation for Java*: Based on Java language features, we focus on methods related to sensitive operations such as injection vulnerability, filtering, string manipulation, and validation operations. Eventually, we defined 21 ISL instructions capable of representing 47 Java methods, as detailed in Table I. These instructions cover various types and functionalities of functions, along with instructions involving if statements, object creation, and more. Table I lists 21 tokens representing Java code elements in the first column, provides brief explanations of each token's meaning in the second column, and lists the code elements they represent in the third column. The fourth column defines the tainted states of each token used in constructing corpora or conducting state marking (see Section III-C3). Certain tokens have explicit states due to their inherent nature, such as always tainted (e.g., *input*) or always untainted (e.g., *contentchk*), while the states of other tokens depend on the context of the instruction sequence.

In Table I, different tokens can express different meanings. For example, *var*, *listVar*, *mapVar*, and *enumVar*, respectively, represent string variables, list variables, dictionary variables, and enum variables. For data of types *List*, *enum*, and *Map*, different order of addition or different ways to retrieving data may produce different effects. Fig. 5 are two code examples about adding and retrieving data from a list in the context of an XSS vulnerability. We assume that the state of "*param*" is tainted. In the code snippet "*valuesList.get(0)*;", if we retrieve the first value, then the state of *var* will be tainted; whereas in the code snippet "*valuesList.get(1)*;", if we retrieve the second value, then the state of *var* will be untainted. Similarly, for variables of type *Map*, not recording keys and values can also impact the propagation of tainted states.

TABLE I
ISL INSTRUCTIONS FOR JAVA

Token	Description	Methods	Taint?
input	input entry	readLine, getString, getParameter, getQueryString, getProperty, getCookie, getAttribute	Y
ss	sensitive sink	sendError, println, execute, addBatch, executeBatch, executeQuery, executeUpdate, exec, addCookie, evaluate, format, setAttribute	Y/N
cond	If instruction		N
chk_num	check number	abs, isCreatable	N
var	var type of String		N
constant	Fixed variable		N
SanitFunc	sanitization function	.prepareStatement+setString, URLEncoder.encode, escapeSql, encodeForCSS, encodeForHTML, encodeForJavaScript, escapeHtml	N
typeFunc	functions for variable types	parseInt, parseLong, parseDouble	
contentchk	Content checking function	equalsIgnoreCase, equals	
nullstr	Initialization string	""	N
strFunc	string function	split, replace, append, trim, replaceAll, substring	Y
operator		==, !=, >, >=, <, <=, &&,	
conc	concatenation operator		Y/N
newClass		class	
func	Function	getValue, read, prepareCall, nextElement, get, add, put, remove	Y/N
object	Class object		Y/N
condVar	Condition var	true, false	N
listVar	Var type of list		Y/N
mapVar	Var type of map		Y/N
enumVar	Var type of enumeration		Y/N
vulType	Vulnerability type		

In addition to variable instructions, we introduce several other types of instructions. For example, in Table I, the `cond` instruction corresponds to if statements, which may include validation methods as part of the conditional expression to validate variables. We also define the `constant` instruction to represent variables of constant types, typically immutable variables. In addition, we have the `newClass` instruction for defining class object operations, as tainted variables may propagate through being passed to newly created objects. Lastly, we have the `strFunc` instruction for string manipulation methods, which may alter the tainted state of strings. The definition and use of these instructions aim to better describe and analyze vulnerability situations in Java code. Furthermore, we introduce the `ss` instruction representing output methods in Java Web, such as database execution or query operations. The use of these instructions enables us to more accurately represent and handle vulnerability situations in Java code.

Finally, to achieve multiclassification of vulnerabilities, we additionally introduce an instruction called `vulType`. This instruction resides at the last position of the intermediate language instruction sequence, and its state represents the type of vulnerability corresponding to the intermediate expression sequence. By introducing this `vulType` instruction, we can better classify and label vulnerabilities, allowing our model to accurately predict and differentiate between different types of vulnerabilities.

2) *ISL Representation for PVCS*: To reserve more outstanding features for vulnerability classification, we retained the method names of output methods. For example, in the SQL injection code shown in Fig. 5, for the statement “`pstmt.executeUpdate()`”, where `executeUpdate` is a method for executing a SQL statement, it can serve as a feature of SQL injection vulnerability. In addition, we also retained the method names for operations, such as adding and deleting variables, such as `List`, `Map`, and `Enum`, as different method names will have different effects on subsequent results. The conversion rules defined by the ISL are specified by the rules in Table I. According to these rules, Java code slices are converted into intermediate

Algorithm 2: ISL Instruction Transformation Algorithm.

Input: *PVCS* // Java code slice

Output: *tokenList* // ISL instruction sequence

```

1: Initialization:
2:   param: var | listVar | mapVar | enumVar
3:   object: java object
4:   method: input | typeFunc | contentchk | chk_num |
   sanitFunc | ss | strFunc
5: Begin
6:   Foreach type of statement in PVCS
7:     Switch(type):
8:       Case: input {input param}
9:       Case: sensitive_sink {ss param}
10:      Case: sanitization {sanitFunc param}
11:      Case: validation {(typeFunc | contentchk |
chk_num) param}
12:      Case: assignment {(param | object) statement}
13:      Case: create object {newClass param}
14:      Case: cond {cond statement cond}
15:      Case: concat {(statement | param) conc
(statement | param)}
16:      Case: str_mod { strFunc param}
17:     End Switch
18:     tokenList ← param.replaceby
(varType_paramName)
19:     tokenList ← object.replaceby (object_objectName)
20:     tokenList ← method.replaceby
(methodType_methodName)
21:   End Foreach
22:   tokenList ← vulType
23: End

```

expression sequences, as illustrated by the specific process in Algorithm 2.

In Algorithm 2, lines 2 to 4 define various variable parameters, objects, and methods. From lines 5 to 17, the algorithm iterates

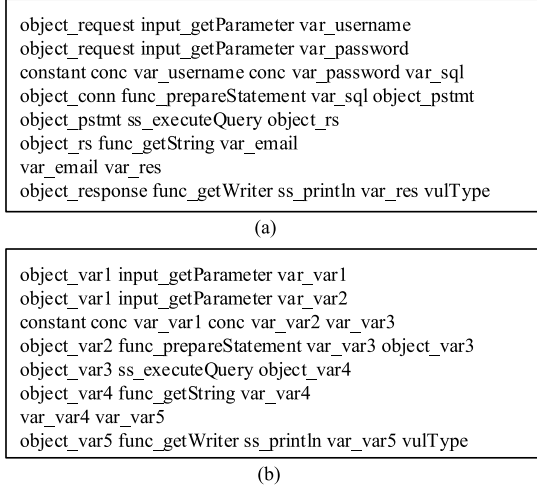


Fig. 6. ISL instruction sequence for PVCS in Fig. 4. (a) ISL instruction sequence for PVCS. (b) Symbolic processing for function and variable names.

through all statements in *PVCS*, translating them into tokens from Table I. Then, from lines 18 to 20, it modifies the form of variables, objects, and methods in each statement. Ultimately, this process generates the ISL instruction sequence for the target *PVCS*. According to *Algorithm 2*, we can convert the slice code in Fig. 4 into the ISL instruction sequence, as shown in Fig. 6(a).

In Fig. 6(a), the statement *String param = request.getParameter("username")* results in the variable *param* being concatenated with its type to become *var_param*, since the *var* instruction represents a string-type variable. The object *request* is concatenated with its type to become *object_request*. Referring to Table I, the method *getParameter* is concatenated with its type to become *input_getParameter*.

Moreover, in order to preserve context-relevant information, we recognize the importance of information within variable names for identifying vulnerability types, as they are crucial for extracting advanced features required by deep neural networks. In the context of deep learning-based vulnerability type identification, we adopted a unified naming convention for function and variable names for symbolic processing, as illustrated in Fig. 6(b). Specifically, we represent variable names as *var0*, *var1*, and so forth, incrementally numbered in sequence. For example, when processing the statement “*String username = request.getParameter('username')*,” we first tag it as a variable assignment statement, where a method call’s return value is assigned to a variable, using *var* as the token for the variable username. Then, we assign a number to the variable name, generating the complete token as “*var_var1*.” If the same variable is encountered again in subsequent sequences, we use the same token for substitution. For variables of data types, such as object, List, and Map, we employ similar processing methods.

3) *State Annotation for PVCS*: In Fig. 6, the content is difficult to understand manually as it represents an ISL instruction sequence. In the context of natural language processing (NLP), sequence models are commonly employed for classifying observed outcomes. In our scenario, these observed outcomes correspond to consecutive instructions in a sequence. Therefore, in this section, we will define the states of these instructions and clarify the transition relationships between them. These states are crucial for our vulnerability classification task as they capture

TABLE II
STATE DEFINITION AND DESCRIPTION

State	Description	Tokens
N-Taint	not tainted	conc,cond,input, var,constant,ss
Taint	tainted	input,var,conc
San	Sanitization	input,sanitFunc, var
Val	validation	typeFunc,var, contentchk
Chg	change string	Input,strFunc,var
Conf	config	Var,func,constant
sqlVul	Sql injection vulnerability	vulType
xssVul	XSS vulnerability	vulType
xpathVul	XPath injection vulnerability	vulType
commandVul	Command injection vulnerability	vulType
noVul	Benign	vulType

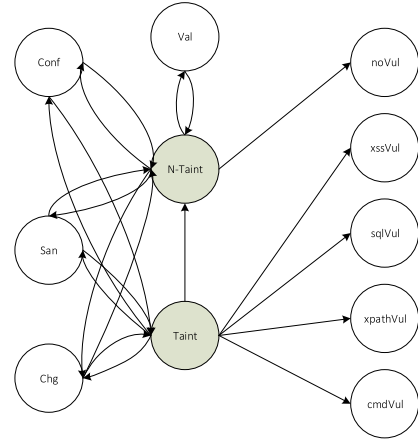


Fig. 7. State transferring.

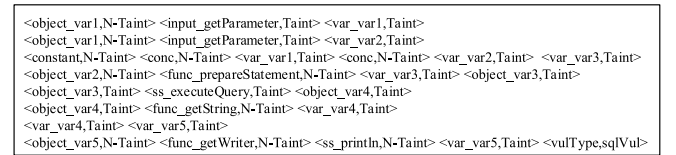


Fig. 8. Example of state annotation process.

contamination situations and variable state changes during code execution.

Consequently, we annotated the states of ISL instruction sequences by defining 11 state instructions as shown in Tables II. Each ISL instruction sequence represents a complete vulnerability triggering process, thus the final state of each sequence can be either tainted (*Taint*) or nontainted (*N-Taint*). In addition, we appended an instruction named *VulType* at the end of ISL instruction sequences for specific vulnerability classification, defining five corresponding vulnerability classification states. However, in the case of web injection vulnerabilities, due to the criticality of string operations, we also need to consider user input and operations related to filtering (*San*), validation (*Val*), configuration (*Conf*), and modification (*Chg*) of relevant variables. Therefore, these four operations are also represented as intermediate states. The process of state model transition is illustrated in Fig. 8.

The case illustrated in Fig. 8 depicts the process from ISL instruction sequence to sequence state annotation, while defining the following state annotation rules.

- 1) The state of input always remains *Taint*. This is because input data, such as `request.getParameter("username")` in Listing 1, may contain potential malicious content or unverified information before any processing.
- 2) The tainted variables, after undergoing sanitization, validation, and string alteration methods, will transition to one of the following three states.
 - When a tainted variable undergoes sanitization, its state changes to *San*.
 - When a tainted variable undergoes validation, its state changes to *Val*.
 - When a tainted variable undergoes string alteration, its state changes to *Chg*.
- 3) For conditional statement code blocks, the variable state initially depends on the state of that variable in the condition expression. For example, in an *if* statement, if the variable *username* changes to the state *Val* after validation, and the contents of the *if* block can only execute after *username* is validated, then the initial state of *username* in the code block is also *Val*.
- 4) The final state of the sequence is always either *noVul* or one of the four vulnerability states. For example, from the last line of code in Fig. 5 and its corresponding final intermediate representation token in Fig. 6(b), based on this rule and the vulnerability states in Table II, the final state of the sequence *sqlVul* is determined to be *sqlVul*.
- 5) Intermediate states *Val*, *San*, and *Chg* appear only in the middle stages of the sequence. For example, in Fig. 6, the instruction sequence begins with an object *object_var1* and ends with a *vulType* token, with states, such as *Val*, *San*, and *Chg*, occurring solely within the middle portion of the sequence.
- 6) The state among variables is transitive. For instance, in Fig. 6(b), the state *Taint* of *var_var5* is transferred to *var_var4* through the assignment operation on line 18 in Fig. 4 of PVCS.

Ultimately, each PVCS will form an instruction sequence represented as a set of $\langle \text{Token}, \text{State} \rangle$ pairs, creating a corpus of injection vulnerability. Fig. 8 is the state annotation sequence obtained from the ISL sequence in Fig. 6(b) according to the above six rules, which will eventually be used as the input of the deep learning model.

D. Vulnerability Detection

After obtaining a large number of vulnerability state annotation sequences, we constructed an injection vulnerability detection model based on BERT-BiLSTM-CRF. The architecture of the model is illustrated in Fig. 9.

1) *BERT*: BERT's Transformer-based encoder captures semantic relationships through token, segment, and position embeddings, modeling context with self-attention. Each input sequence starts with a [CLS] token and ends with a [SEP] token, helping BERT focus on rich contextual features for vulnerability detection.

2) *BiLSTM-CRF*: BiLSTM generates a contextual representation for each word by processing forward and backward dependencies, which is then passed to a CRF layer. The CRF

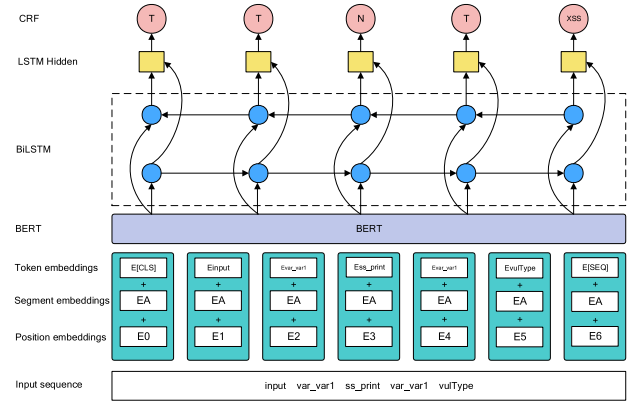


Fig. 9. BERT-BiLSTM-CRF model.

TABLE III
PARAMETER SET FOR BERT-BiLSTM-CRF MODEL

	parameter name	value
Bert-base-cased	network layers	12
	Hidden layer	768
	Self attention head	12
	parameter	110M
BiLSTM-CRF	Embedding Dimension	768
	Iteration Times	20
	Sequence Length	200
	Optimizer	Adam
	Learning Rate	0.00001
	Hidden Layer Dimension	768
	Dropout Rate	0.1

optimizes label sequences by considering dependencies between labels.

In summary, the BERT-BiLSTM-CRF model combines BERT's deep semantic understanding with BiLSTM-CRF's sequence labeling for effective vulnerability detection. Using the BERT-base-cased model, we fine-tuned later layers and updated the vocabulary to include domain-specific instructions. Processed data is tokenized into tensor vectors, with the final token in each sequence, marked as *vulType*, representing the vulnerability detection result. Model parameters are listed in Table III.

IV. EXPERIMENTS AND RESULTS

The configuration and environment for this experiment are as follows: the operating system is Ubuntu 20.04, equipped with an Intel(R) Xeon(R) Gold 6348 CPU@2.60 GHz, boasting 1 TB of RAM, and configured with 4 * RTX A40 GPUs. On the software side, we are using the PyTorch 1.2 deep learning framework, programming in Python 3.9, and developing within the Anaconda 3 virtual environment.

1) *Datasets*: We select two well-known open-source projects from the community, namely, the Juliet Test Suite [44] and the WASP Benchmark [46], as our testing datasets to evaluate the robustness and accuracy of IVIB across different scenarios. Both of these publicly available datasets are developed in Java. The Juliet Test Suite contains various types of vulnerable code covering common security vulnerabilities such as buffer overflow, code injection, etc. The WASP Benchmark

TABLE IV
VULNERABILITY CLASSIFICATION AND THE NUMBER OF CODE SLICES AND TOKENS

Data set	SQL				XSS				XPATHI				CMDI				Total			
	Slices		Tokens		Slices		Tokens		Slices		Tokens		Slices		Tokens		Slices		Tokens	
Benchmark [46]	vul	benign	vul	benign	vul	benign	vul	benign	vul	benign	vul	benign	vul	benign	vul	benign	vul	benign	vul	benign
Juliet [44]	12	44	342	1215	20	33	462	594	42	49	1804	5944	18	33	477	663	92	259	3085	8416
Sum	287	264	9245	8856	268	186	9370	6407	57	69	2225	6506	109	120	3662	4075	721	739	24502	25844
Total	551		18101		454		15777		226		8731		229		7737		1460		50346	

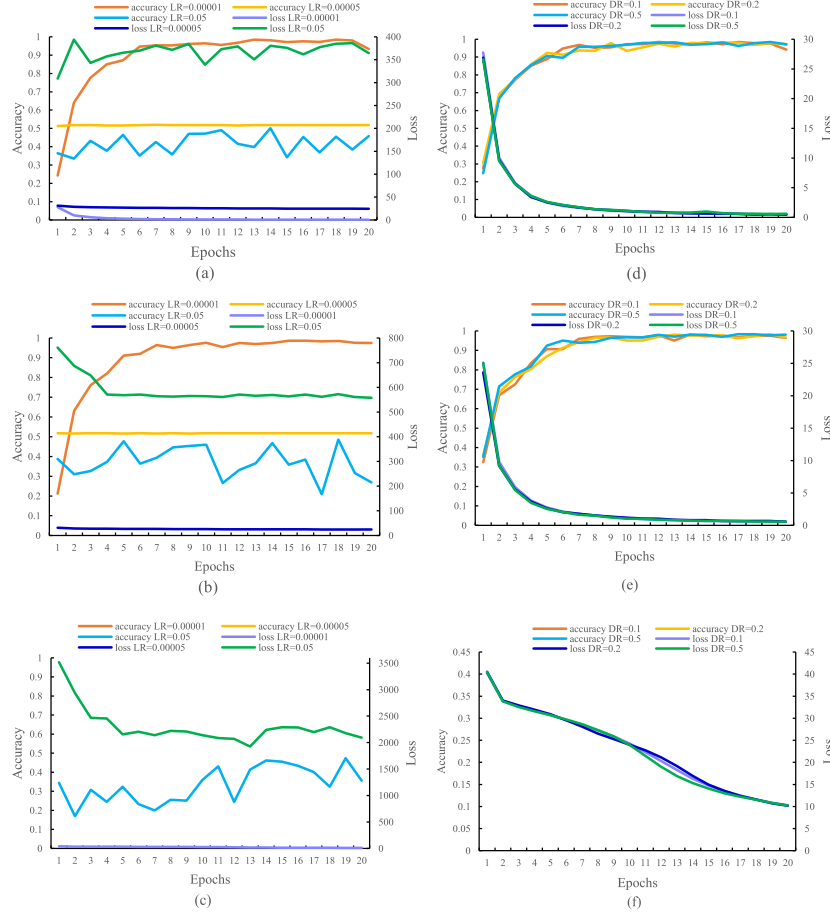


Fig. 10. Hyperparameters training results. (a) Adam and learning rate. (b) RMSprop and learning rate. (c) SGD and learning rate. (d) Adam and dropout rate. (e) RMSprop and dropout rate. (f) SGD and dropout rate.

is another popular open-source vulnerability testing suite focusing on security vulnerabilities in web applications, encompassing various common web vulnerabilities including XSS, SQL injection, etc. The detailed classification of vulnerabilities and the number of code slices and tokens are shown in Table IV.

2) *Optimal Parameter Tuning for Model Performance*: To achieve optimal model performance and identify suitable hyperparameters (learning rate, dropout rate, and optimizer), we used 10-fold cross-validation with an 80/20 train-test split. Key hyperparameters were tuned, including learning rates (0.00001, 0.00005, 0.05) and optimizers (Adam, RMSprop, SGD). Standardization and regularization settings, particularly dropout rates (0.1, 0.2, 0.5), were applied to prevent overfitting and improve generalization. As shown in Fig. 10, the best results were obtained with the Adam optimizer (learning rate 0.00001, dropout rate 0.1) over 20 epochs with 768 neurons per hidden layer.

TABLE V
ABLATION EXPERIMENT RESULTS

Model	Accuracy	Precision	Recall	F1
BERT-softmax	90.10%	87.29%	90.10%	88.23%
BERT-CRF	95.22%	95.32%	95.22%	95.21%
BiLSTM-CRF	87.37%	82.32%	87.37%	84.43%
BERT-BiLSTM-CRF	96.93%	97.00%	96.93%	96.93%

The bold values represent the maximum value in each column (the best-performing metric).

3) *Ablation Experiment Results*: The ablation experiments aim to assess the contribution of different model components to the overall performance in vulnerability detection, ultimately guiding the choice of BERT-BiLSTM-CRF as the optimal configuration. The results, shown in Table V, demonstrate that BERT-CRF achieves an F1-score of 95.21%, surpassing BERT-softmax, which scores 88.23%. This outcome highlights CRF's ability to handle label dependencies effectively by modeling

TABLE VI
DIFFERENCES AMONG IVIB AND OTHE SOTA METHODS

	Model	Data Processing	Procedural	Granularity
Saccente et al. [28]	LSTM	java tokens	Intra-	Word level
Kim et al. [37]	BERT	code gadgets	Inter-	Sentence level
Ziems et al. [14] (M1)	BERT	java tokens	Intra-	Word level
Ziems et al. [14] (M2)	BERT-BiLSTM	java tokens	Intra-	Word level
IVIB	BERT-BiLSTM-CRF	ISL tokens	Inter-	Word level

global relationships across label sequences, which enhances the model's capacity to capture contextual information in sequential data.

In comparison, the BiLSTM-CRF model achieves an F1-score of 84.43%, underscoring the importance of BERT's contextual language understanding, which BiLSTM-CRF lacks. This gap illustrates that without BERT's language comprehension, the model's ability to grasp deeper context is limited.

Building upon these findings, our proposed BERT-BiLSTM-CRF model achieves an F1-score of 96.93%, combining BERT's robust feature extraction and BiLSTM's effectiveness in capturing long-term dependencies. This configuration outperforms BERT-CRF and BERT-softmax, confirming that the integration of BiLSTM further enhances feature extraction on top of BERT. These results validate BERT-BiLSTM-CRF as the most effective setup for precise vulnerability detection.

4) *Comparison Results With the SOTA Methods*: For an in-depth comparative study, we selected a series of SOTA vulnerability detection methods and compared them in our experiments. These methods are summarized in Table VI as follows.

The results of the comparison with the four methods are shown in Table VII. We utilized four types of vulnerabilities: SQL injection, XSS injection, XPath injection, and Command injection, to evaluate the respective evaluation metrics of these methods.

The results show that our method outperforms the other four methods in detecting SQL injection, XSS injection, and CMD injection vulnerabilities. However, for XPath injection vulnerabilities, our method does not perform as well as the other three methods except for Saccente et al. [28]. Upon analysis, it was found that Juliet's slice and token counts dominate in XPath injection vulnerabilities. Compared to the Benchmark dataset, the Juliet dataset exhibits higher code repetition. For instance, in the source files *CWE89_SQL_Injection_getParameter_Servlet_executeQuery_04.java* and *CWE89_SQL_Injection_getParameter_Servlet_executeQuery_05.java* in the Juliet dataset, the only difference in the bad method is the variable name, while their code structures are identical, as shown in Fig. 11. Therefore, after code slicing, a large amount of redundant code is introduced, which affects our model's ability to extract features [47]. In contrast, the IVIB model is more suitable for applications with significantly different feature data, exhibiting stronger generalization capabilities.

In addition, to assess IVIB's computational overhead, we calculated the average time for model training and detection over 20 epochs. This approach provides a comprehensive view of time consumption, particularly as the model approaches convergence. Results in Table VII indicate that IVIB has the highest time cost among the methods, averaging just under 90 s, due to the integration of BERT, BiLSTM, and CRF. The word-level approach in IVIB indeed increases computational overhead by processing a large number of tokens, which extends runtime. Other BERT-based methods also exceed 50 s, placing them in

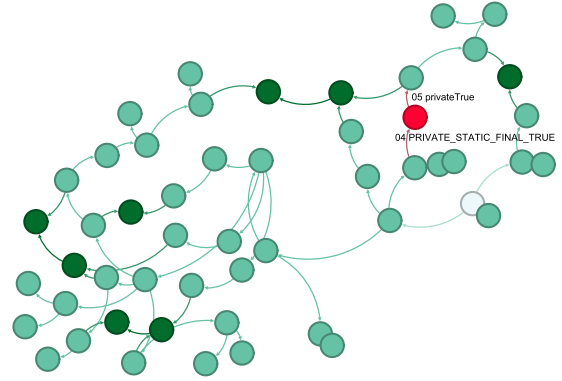


Fig. 11. Node structure of ... *executeQuery_04.java* and ... *executeQuery_05.java*.

a similar range. In contrast, the method by Saccente et al. [28], which does not use BERT, has the lowest time overhead at under 3 s, though its accuracy and F1-score are considerably lower. We believe that the additional computational cost of IVIB is justified by its high performance, making this overhead worthwhile for high-stakes applications that require precision.

5) *Detection Results in Real World Projects*: To assess the effectiveness of IVIB in real-world projects, we conducted tests on IVIB based on four actual projects from GitHub: JavaVulnerableLab [48], Dwa [49], Vulnerable-Web-App [50], and Java_DVWA [51]. We will utilize these projects to evaluate our model, and the specific detection results are presented in Table VIII.

- 1) In the JavaVulnerableLab [48] project, we successfully detected all injection vulnerabilities in 10 slices, including 5 SQLIs, 2 XSS, and 1 XPATHI. IVIB did not produce any false positives or false negatives for these slices.
- 2) In the Dwa [49] project, we found 10 vulnerabilities in 12 slices, including 8 SQLIs, 1 XSS, and 1 CMDI. There were 2 false positives in the results, where 1 XSS was predicted as XPATHI, and 1 nonVul was predicted as XSS.
- 3) In the Vulnerable-Web-App [50] project, we detected a total of 7 real vulnerabilities in 14 slices, including 5 SQLIs, 1 XSS, and 1 CMDI. IVIB generated 3 false positives in these slices, predicting 3 nonVul as XSS.
- 4) In the Java_DVWA [51] project, we analyzed 6 slices and detected 3 real vulnerabilities, including 2 SQLIs and 1 CMDI. IVIB produced 2 false positives in these results, predicting noVul and XSS as SQLIs.

In total, 42 vulnerability cases were used across these projects, comprising 30 slices with injection vulnerabilities and 12 slices with noninjection vulnerabilities. Eventually, 28 injection vulnerabilities and 7 non-injection vulnerabilities were detected, resulting in an overall accuracy of 83.33%, a recall of 100%, a precision of 80%, and an F1-score of 88.9%. This demonstrates that IVIB performs well in practice.

Moreover, we have also evaluated the inter-procedural method proposed by Kim et al. [37] on these four projects, but it failed to predict correct results. This is because their approach utilizes lines of code as vector dimensions, yet significant differences in coding styles across various projects may exist. Such variations pose challenges for BERT to accurately capture, potentially resulting in diminished predictive performance. For instance, considering the following two coding styles: “int a = 1; int

TABLE VII
COMPARISON RESULTS WITH THE SOTA METHODS

	SQLI			XSS			XPATHI			CMDI		
	Accuracy	F1-score	Time(s)	Accuracy	F1-score	Time(s)	Accuracy	F1-score	Time(s)	Accuracy	F1-score	Time(s)
Saccante et al. [28]	48.18%	31.3%	2.75	59.34%	44.2%	2.72	23.91%	9%	2.67	54.35%	38.2%	2.67
Kim et al. [37]	69.09%	68.38%	53.0	71.43%	68.26%	52.97	93.47%	93.69%	52.57	58.69%	58.56%	52.57
Ziems et al. [14] (M1)	67.27%	62.14%	51.76	68.81%	69.29%	51.6	93.47%	93.10%	51.28	50%	50.5%	51.28
Ziems et al. [14] (M2)	69.09%	64.82%	71.17	54.84%	54.84%	70.38	89.13%	90.03%	69.98	67.39%	68.5%	69.92
IVIB	80.18%	82.8%	89.1	70.65%	73.8%	88.88	86.66%	88.1%	88.33	67.39%	70.6%	88.31

TABLE VIII
DETECTION RESULTS FOR IVIB IN REAL WORLD PROJECTS

	Slices	SQLI		XSS		XPATHI		CMDI		noVul		FP	FN	TP	TN
		ori	pred	ori	pred	ori	pred	ori	pred	ori	pred				
JavaVulnerableLab [48]	10	5	5	2	2	1	1	0	0	2	2	0	0	8	2
Dwa [49]	12	8	8	2	2	0	1	1	1	1	0	2	0	10	0
Vulnerable-Web-App [50]	14	5	5	1	4	0	0	1	1	7	4	3	0	7	4
Java_DVWA [51]	6	2	4	1	0	0	0	1	1	2	1	2	0	3	1
Total	42	20	22	6	8	1	2	3	3	12	7	7	0	28	7

b = 2;” and “int a=1, b=2;”, while both expressions convey the same meaning, the model, processing code on a per-line basis, may struggle to discern subtle coding style differences. Conversely, IVIB, during code parsing, can account for the structure, syntax, and semantic information of the code at the variable and expression levels, thereby ensuring the reliability of predictive outcomes.

V. RELATED WORK

The studies on web injection vulnerabilities has achieved extensive results, covering various aspects such as static analysis, dynamic detection, hybrid detection, Deep learning methods based on static or dynamic features.

A. Static Analysis

Static analysis methods typically inspect a program’s source code or bytecode to identify potential injection points and vulnerability characteristics. Huang et al. [3] proposed a static analysis method for detecting SQL injection and XSS vulnerabilities in PHP web applications. Their approach automatically adds runtime protections to insecure code sections, immediately strengthening PHP code postprocessing. Although it efficiently manages insertion quantity to reduce overhead, its reliance on intraprocedural analysis restricts modeling to method boundaries. Fu et al. [4] developed a static analysis framework to pinpoint SQL injection vulnerabilities during compilation. By parsing ASP.NET web app Microsoft Intermediate Language bytecode, it employs symbolic execution and constraint solvers to scrutinize backend code iteratively, handling intricate string constraints. However, it necessitates building an attack pattern library using regular expressions, heavily dependent on human expertise, despite its effectiveness in detection. Fiallos et al. [5] presented a validation filter scheme utilizing Java, OWASP Stinger, and JBoss Wildfly. This filter rigorously validates and filters all incoming network requests, considering aspects, such as data types, lengths, and regular expressions. Yet, creating regexes for known vulnerabilities is necessary, indicating potential limitations in detecting new vulnerabilities, relying heavily on human expertise.

In our work, IVIB stands out by employing interprocedural source code analysis, effectively mitigating the risk of false positives and negatives arising from incomplete identification

of vulnerable code segments. Furthermore, we developed an intermediate language and state model to tackle the challenge of dependency on security experts’ experience, empowering the system to autonomously acquire insights into vulnerability-related knowledge.

B. Dynamic Analysis

Dynamic analysis [6] detects vulnerabilities by capturing the actual behavior of a program during runtime, with the aim of exposing malicious activities executed by the executable file at runtime. Chess and West [7] proposed a dynamic taint-based method for vulnerability detection, validating inputs before utilizing tainted data during program execution. However, dynamic analysis has constraints, detecting vulnerabilities solely in executed paths and being unable to uncover flaws in unexecuted code. Furthermore, it cannot ascertain if the input set covers all potential program execution scenarios. He and He [8] devised a Java Web app SQL injection detection method leveraging aspect-oriented programming (AOP) and dynamic taint analysis. They encapsulated taint analysis stages (marking, propagation, SQL syntax) via AOP, forming an aspect library for reuse. Yet, addressing large or intricate applications reveals coverage and automation shortcomings. Falana et al. [9] enhanced XSS attack detection in web apps via dynamic analysis and a fuzzing engine. They employed a web crawler to identify all application entry points (AEPs) and generated invalid strings for each AEP until evading the web app firewall. While maintaining an up-to-date payload database, manual configuration remains necessary initially.

Dynamic analysis suffers from limited coverage, whereas IVIB proposed in our work, employing inter-procedural slicing across various hierarchical levels, comprehensively covers all code segments. By reusing parsed code, IVIB enhances both efficiency and accuracy.

C. Static and Dynamic (hybrid) Analysis

The hybrid analysis method combines the advantages of static analysis and dynamic analysis to discover potential vulnerabilities in applications and improve detection speed and efficiency. Monga et al. [10] proposed Phan, a hybrid analysis framework for detecting web application vulnerabilities. It operates at the bytecode level rather than on PHP source code, conducting

static analysis initially to identify potential vulnerabilities. During dynamic analysis, it focuses solely on monitoring these identified statements. However, professional security analysts are needed for designing and implementing processes, such as feature extraction and rule formulation. Shar et al. [11] developed PhpMiner, a tool for predicting web application vulnerabilities, including XSS, SQL injection, remote code execution, and file inclusion vulnerabilities. This method integrates static and dynamic analyses, with static analysis computing receiver slices and dynamic analysis inferring sanitization and validation function types. Supervised and semi-supervised predictors are then constructed to forecast network vulnerabilities. However, due to the challenge of handling numerous features and samples, it suffers from the curse of dimensionality and yields a notable false positive rate in predicting XSS vulnerabilities.

The combination of static analysis and dynamic analysis in hybrid analysis requires a significant amount of manual intervention to design and implement processes, such as feature extraction and rule formulation, and is susceptible to the curse of dimensionality when dealing with a large number of features and samples. The BERT-BiLSTM-CRF vulnerability detection model proposed in this article can automatically conduct feature learning and address the curse of dimensionality by combining intermediate language parsing.

D. Deep Learning Methods Based on Static or Dynamic Features

Deep learning has been widely applied across security domains, including web vulnerability detection [12], [13], [26]. Early research in this field leveraged LSTM models, as seen in Saccente et al. [28], who developed LSTM-based networks to handle various common vulnerabilities and exposures (CVE) vulnerabilities, each model dedicated to binary classification. However, these models encountered gradient issues with long sequences, impacting stability and accuracy. Fidalgo et al. [29] employed an intermediate language for representing PHP code in LSTM models to detect SQL injection but faced challenges in handling long-term dependencies and complex structures.

To improve upon LSTM's limitations, BiLSTM gained attention [27] for its ability to capture both forward and backward information, alleviating gradient issues common in traditional LSTM. Li et al. [23] introduced VulDeePecker, a BiLSTM-based model using code gadgets vectorized with word2vec, and later developed mVulDeePecker [24], which extended it to a multi-class vulnerability detector. Zhang et al. [25] proposed ISVSF, a Java vulnerability detection framework using BiLSTM and control flow trees. Vulhunter [30] applied BiLSTM to bytecode features for SQL and XSS detection but struggled with long-term dependencies in lengthy sequences.

As researchers sought improved feature representation before detection, alternatives to word2vec emerged. Gogoi et al. [32] used word2vec in SQL injection detection, while Watson et al. [33] explored Sent2Vec for contextual semantics but found it limited in fully capturing function-level context. BERT, with its capacity for nuanced semantic understanding, has since become a popular choice. Luo et al. [35] compared BERT with other embeddings, finding it performed best, even under adversarial samples. Mamede et al. [36] explored a BERT-based multilabel classifier for Java vulnerabilities, while Kim et al. [37] developed VulDeBERT, a BERT-powered tool that fine-tunes

code snippets to detect specific vulnerabilities effectively. Specially, Kim et al. [37] primarily employ sentence-level interprocedural analysis focused on function-level connections through system calls, their approach lacks the fine-grained dependency tracking across classes required for Java-based applications. This limitation is notable, as it restricts the model's ability to capture class-level relationships, particularly in object-oriented code. In contrast, IVIB's approach integrates intraprocedural and interprocedural dependencies across both methods and classes, providing a comprehensive view of Java's object-oriented structure and enhancing adaptability for diverse vulnerabilities, such as SQL injection and XSS.

Currently, there are also deep learning methods for dealing with vulnerabilities in Java code. Feng et al. [38] developed a graph neural network classifier for assessing Java program maliciousness, but their reliance on static analysis techniques may overlook critical control dependencies and data transformations, leading to incomplete vulnerability detection. Chi et al. [39] focused on automatic vulnerability fixing using neural machine translation techniques, which primarily address remediation rather than proactive detection. Nguyen et al. [40] introduced MiDas for identifying vulnerability-fixing commits, yet its effectiveness can be hampered by the noise in commit data, potentially affecting detection accuracy. Song and Ding [41] explored the robustness of deep learning models against adversarial attacks, neglecting the direct application of these models for vulnerability detection. Liu et al. [42] proposed GraphSearchNet to improve code search functionality, but this focus does not provide a comprehensive solution for identifying vulnerabilities. In contrast, IVIB incorporates a robust framework for interprocedural analysis, capturing both data and control flow dependencies, and achieves high accuracy in vulnerability detection.

Building upon these insights, our work combines BERT and BiLSTM with a CRF layer [34] to form the BERT-BiLSTM-CRF model. This model leverages BERT's understanding of contextual semantics, while BiLSTM-CRF not only captures dependencies between adjacent code features but also mitigates gradient vanishing and exploding issues. By extracting rich feature representations from BERT's output for optimal label sequence prediction, IVIB achieves robust and accurate vulnerability detection.

VI. DISCUSSION

The development of IVIB represents a significant advancement in Java Web application security, particularly in the realm of injection vulnerability detection. The core to IVIB is the modeling of a SDG that captures various dependencies—control, data, method, and class—providing a comprehensive framework to identify complex vulnerability patterns in Java's object-oriented structure. This approach is particularly beneficial for enterprise-level applications, where Java's complexity and widespread use necessitate a robust detection framework. IVIB also introduces a novel ISL specifically tailored to Java, incorporating elements, such as classes and objects, that are essential for capturing Java's unique structural characteristics. This precise representation supports IVIB's deep learning model by providing a more accurate depiction of code, thereby enhancing the detection process. At the another core of IVIB's model is the BERT-BiLSTM-CRF architecture. This advanced combination leverages BERT's ability for deep contextual understanding, BiLSTM's management of long-range dependencies, and CRF's

sequence labeling optimization, enabling IVIB to achieve high accuracy in vulnerability identification.

Furthermore, IVIB expands its detection capabilities beyond traditional SQL injection to encompass multiple vulnerabilities, including XSS, CMDI, and XPATHI. This broad scope ensures that IVIB is well-suited to address diverse security challenges in real-world Java applications.

Evaluation on diverse datasets, including both open-source and real-world projects, validates the effectiveness and practicality of IVIB. Notably, comparison with state-of-the-art methods reveals IVIB's superior performance in accuracy and generalization.

However, IVIB still has some limitations. When facing datasets with high code similarity, we plan to explore more advanced embedding techniques or feature engineering to distinguish similar code segments. Second, in the process of building the corpus, we rely on manual annotation, and in the future, we will explore automatic annotation methods to improve efficiency and consistency, leveraging NLP techniques. Third, we plan to train a specialized BERT model based on a large amount of Java Web source code to further enhance the capture of Java code semantics and contextual information, improving vulnerability detection effectiveness.

VII. CONCLUSION

In this article, we proposed a prototype tool, IVIB, a Java Web application injection vulnerability detection method based on interprocedural analysis and the BERT-BiLSTM-CRF model. IVIB represents a significant step forward in Java web application security, offering a robust and efficient method for injection vulnerability detection. Through the integration of interprocedural analysis and the BERT-BiLSTM-CRF model, IVIB achieves precise and efficient identification of vulnerability locations, even in the face of complex code structures. Comparison with state-of-the-art methods demonstrates that IVIB performs well in most cases, with significant advantages in accuracy and generalization performance. Finally, IVIB is conducted vulnerability detection on multiple real-world projects and achieved notable results, accurately detecting 28 vulnerabilities out of 30 vulnerable slices and 7 benign slices, with low false positives and zero false negative.

While IVIB has demonstrated notable effectiveness, there remain areas for improvement. Future research will explore advanced embedding techniques or feature engineering to address datasets with high code similarity, as well as automated annotation methods to enhance efficiency and consistency in corpus building. In addition, the development of a specialized BERT model trained on a large corpus of Java Web source code holds promise for further improving vulnerability detection effectiveness by capturing nuanced semantics and contextual information. Overall, IVIB stands as a promising tool in the ongoing effort to secure Java Web applications against injection vulnerabilities, with potential for continued refinement and expansion in the future.

REFERENCES

- [1] OWASP, "OWASP TOP 10," 2021. [Online]. Available: <https://owasp.org/Top10/>
- [2] JetBrains, "The state of developer ecosystem 2022," 2022. [Online]. Available: <https://www.jetbrains.com/lp/devecosystem-2022/>
- [3] Y.-W. Huang, F. Yu, C. Hang, C.-H. Tsai, D.-T. Lee, and S.-Y. Kuo, "Securing web application code by static analysis and runtime protection," in *Proc. 13th Int. Conf. World Wide Web*, 2004, pp. 40–52.
- [4] X. Fu, X. Lu, B. Peltsverger, S. Chen, K. Qian, and L. Tao, "A static analysis framework for detecting SQL injection vulnerabilities," in *Proc. 31st Annu. Int. Comput. Softw. Appl. Conf.*, 2007, pp. 87–96.
- [5] S. I. Fiallos, J. B. Higuera, M. Intriago-Pazmiño, J. R. B. Higuera, J. A. S. Montalvo, and J. Cubo, "Effective filter for common injection attacks in online web applications," *IEEE Access*, vol. 9, pp. 10378–10391, 2021.
- [6] T. Ball, "The concept of dynamic analysis," *ACM SIGSOFT Softw. Eng. Notes*, vol. 24, no. 6, pp. 216–234, 1999.
- [7] B. Chess and J. West, "Dynamic taint propagation: Finding vulnerabilities without attacking," *Inf. Secur. Tech. Rep.*, vol. 13, no. 1, pp. 33–39, 2008.
- [8] C. He and Y. He, "A reusable SQL injection detection method for java web applications," *KSII Trans. Internet Inf. Syst.*, vol. 14, no. 6, pp. 2576–2590, 2020.
- [9] O. J. Falana, I. O. Ebo, C. O. Tinubu, O. A. Adejimi, and A. Ntuk, "Detection of cross-site scripting attacks using dynamic analysis and fuzzy inference system," in *Proc. 2020 IEEE Int. Conf. Math., Comput. Eng. Comput. Sci.*, 2020, pp. 1–6.
- [10] M. Monga, R. Paleari, and E. Passerini, "A hybrid analysis framework for detecting web application vulnerabilities," in *Proc. 2009 ICSE Workshop Softw. Eng. Secure Syst.*, 2009, pp. 25–32.
- [11] L. K. Shar, L. C. Briand, and H. B. K. Tan, "Web application vulnerability prediction using hybrid program analysis and machine learning," *IEEE Trans. Dependable Secure Comput.*, vol. 12, no. 6, pp. 688–707, Nov./Dec. 2015.
- [12] M. T. B. Nazim et al., "Systematic analysis of deep learning model for vulnerable code detection," in *Proc. IEEE 46th Annu. Comput., Softw., Appl. Conf.*, 2022, pp. 1768–1773.
- [13] P. Zeng, G. Lin, L. Pan, Y. Tai, and J. Zhang, "Software vulnerability analysis and discovery using deep learning techniques: A survey," *IEEE Access*, vol. 8, pp. 197158–197172, 2020.
- [14] N. Ziems and S. Wu, "Security vulnerability detection using deep learning natural language processing," in *Proc. IEEE INFOCOM 2021-IEEE Conf. Comput. Commun. Workshops*, 2021, pp. 1–6.
- [15] S. Liu et al., "CD-VulD: Cross-domain vulnerability discovery based on deep domain adaptation," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 1, pp. 438–451, Jan./Feb. 2020.
- [16] S. Arakelyan, S. Arasteh, C. Hauser, E. Kline, and A. Galstyan, "Bin2vec: Learning representations of binary executable programs for security tasks," *Cybersecurity*, vol. 4, pp. 1–14, 2021.
- [17] M. F. Rozi, S. Kim, and S. Ozawa, "Deep neural networks for malicious javascript detection using bytecode sequences," in *Proc. 2020 Int. Joint Conf. Neural Netw.*, 2020, pp. 1–8.
- [18] W. Zheng, J. Gao, X. Wu, Y. Xun, G. Liu, and X. Chen, "An empirical study of high-impact factors for machine learning-based vulnerability detection," in *Proc. IEEE 2nd Int. workshop Intell. Bug Fixing*, 2020, pp. 26–34.
- [19] Y. Fang, S. Han, C. Huang, and R. Wu, "Tap: A static analysis model for PHP vulnerabilities based on token and deep learning technology," *PLoS One*, vol. 14, no. 11, 2019, Art. no. e0225196.
- [20] C. Lin, Y. Xu, Y. Fang, and Z. Liu, "Vuleye: A novel graph neural network vulnerability detection approach for PHP application," *Appl. Sci.*, vol. 13, no. 2, 2023, Art. no. 825.
- [21] R. Russell et al., "Automated vulnerability detection in source code using deep representation learning," in *Proc. 17th IEEE Int. Conf. Mach. Learn. Appl.*, 2018, pp. 757–762.
- [22] B. Zhang, J. Li, J. Ren, and G. Huang, "Efficiency and effectiveness of web application vulnerability detection approaches: A review," *ACM Comput. Surv.*, vol. 54, no. 9, pp. 1–35, 2021.
- [23] Z. Li, D. Zou, S. Xu, X. Ou, and Y. Zhong, "Vuldeepecker: A deep learning-based system for vulnerability detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, doi: [10.14722/ndss.2018.23158](https://doi.org/10.14722/ndss.2018.23158).
- [24] D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, "μvuldeepecker: A deep learning-based system for multiclass vulnerability detection," *IEEE Trans. Dependable Secure Comput.*, vol. 18, no. 5, pp. 2224–2236, Sep./Oct. 2021.
- [25] H. Zhang, Y. Bi, H. Guo, W. Sun, and J. Li, "ISVSF: Intelligent vulnerability detection against java via sentence-level pattern exploring," *IEEE Syst. J.*, vol. 16, no. 1, pp. 1032–1043, Mar. 2022.
- [26] R. L. Alaoui and E. H. Nfaoui, "Deep learning for vulnerability and attack detection on web applications: A systematic literature review," *Future Internet*, vol. 14, no. 4, 2022, Art. no. 118.

- [27] W. Zheng et al., "The impact factors on the performance of machine learning-based vulnerability detection: A comparative study," *J. Syst. Softw.*, vol. 168, 2020, Art. no. 110659.
- [28] N. Saccente, J. Dehlinger, L. Deng, S. Chakraborty, and Y. Xiong, "Project achilles: A prototype tool for static method-level vulnerability detection of java source code using a recurrent neural network," in *Proc. 34th IEEE/ACM Int. Conf. Automated Softw. Eng. Workshop*, 2019, pp. 114–121.
- [29] A. Fidalgo, I. Medeiros, P. Antunes, and N. Neves, "Towards a deep learning model for vulnerability detection on web application variants," in *Proc. 2020 IEEE Int. Conf. Softw. Testing, Verification Validation Workshops*, 2020, pp. 465–476.
- [30] N. Guo, X. Li, H. Yin, and Y. Gao, "Vulhunter: An automated vulnerability detection system based on deep learning and bytecode," in *Proc. Inf. Commun. Secur.: 21st Int. Conf., ICICS 2019, Beijing, China, Dec. 15–17, 2019, Revised Sel. Papers 21*, 2020, pp. 199–218.
- [31] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Adv. Neural Inf. Process. Syst.*, 2013, pp. 3111–3119.
- [32] B. Gogoi, T. Ahmed, and A. Dutta, "Defending against SQL injection attacks in web applications using machine learning and natural language processing," in *Proc. IEEE 18th India Council Int. Conf.*, 2021, pp. 1–6.
- [33] A. Watson, E. Ufuktepe, and K. Palaniappan, "Detecting software code vulnerabilities using 2D convolutional neural networks with program slicing feature maps," in *Proc. 2022 IEEE Appl. Imagery Pattern Recognit. Workshop*, 2022, pp. 1–9.
- [34] Q. Li, W. Tang, X. Chen, S. Feng, and L. Wang, "Comprehensive vulnerability aspect extraction," *Appl. Intell.*, vol. 54, no. 3, pp. 2881–2899, 2024.
- [35] Y. Luo, W. Xu, and D. Xu, "Detecting integer overflow errors in java source code via machine learning," in *Proc. IEEE 33rd Int. Conf. Tools With Artif. Intell.*, 2021, pp. 724–728.
- [36] C. Mamede, E. Pinconschi, R. Abreu, and J. Campos, "Exploring transformers for multi-label classification of java vulnerabilities," in *Proc. IEEE 22nd Int. Conf. Softw. Qual., Rel. Secur.*, 2022, pp. 43–52.
- [37] S. Kim, J. Choi, M. E. Ahmed, S. Nepal, and H. Kim, "Vuldebert: A vulnerability detection system using bert," in *Proc. 2022 IEEE Int. Symp. Softw. Rel. Eng. Workshops*, 2022, pp. 69–74.
- [38] P. Feng, L. Yang, D. Lu, N. Xi, and J. Ma, "Bejagmn: Behavior-based java malware detection via graph neural network," *J. Supercomputing*, vol. 79, no. 14, pp. 15390–15414, 2023.
- [39] J. Chi, Y. Qu, T. Liu, Q. Zheng, and H. Yin, "Seqtrans: Automatic vulnerability fix via sequence to sequence learning," *IEEE Trans. Softw. Eng.*, vol. 49, no. 2, pp. 564–585, Feb. 2023, times Cited: 001939-3520.
- [40] T. G. Nguyen et al., "Multi-granularity detector for vulnerability fixes," *IEEE Trans. Softw. Eng.*, vol. 49, no. 8, pp. 4035–4057, Aug. 2023.
- [41] L. Song and S. H. Ding, "Milo: Attacking deep pre-trained model for programming languages tasks with anti-analysis code obfuscation," in *Proc. IEEE 47th Annu. Computers, Software, Appl. Conf.*, 2023, pp. 586–594.
- [42] S. Liu, X. Xie, J. Siow, L. Ma, G. Meng, and Y. Liu, "Graphsearchnet: Enhancing GNNs via capturing global dependencies for semantic code search," *IEEE Trans. Softw. Eng.*, vol. 49, no. 4, pp. 2839–2855, Apr. 2023.
- [43] I. Medeiros, N. Neves, and M. Correia, "Statically detecting vulnerabilities by processing programming languages as natural languages," *IEEE Trans. Rel.*, vol. 71, no. 2, pp. 1033–1056, Jun. 2022.
- [44] "JavaParser," (n.d.). [Online]. Available: <https://javaparser.org/>
- [45] S. Horwitz, T. Reps, and D. Binkley, "Interprocedural slicing using dependence graphs," *ACM Trans. Program. Lang. Syst.*, vol. 12, no. 1, pp. 26–60, 1990.
- [46] P. E. Black and P. E. Black, *Juliet 1.3 Test suite: Changes From 1.2*. US Department of Commerce, National Institute of Standards and Technology, 2018.
- [47] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, "Deep learning based vulnerability detection: Are we there yet?," *IEEE Trans. Softw. Eng.*, vol. 48, no. 9, pp. 3280–3296, Sep. 2022.
- [48] CSPF-Founder, "JavaVulnerableLab," 2015. [Online]. Available: <https://github.com/CSPF-Founder/JavaVulnerableLab>
- [49] Bletvaska, "Dwa," 2017. [Online]. Available: <https://github.com/bletvaska/dwa>
- [50] Nav7neeet, "Vulnerable-web-app," 2021. [Online]. Available: <https://github.com/nav7neeet/Vulnerable-Web-App>
- [51] Ch35Tnut, "Java_DVWA," 2023. [Online]. Available: https://github.com/Ch35Tnut/Java_DVWA